

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5

7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism

Response to Questions	10	Responds accurately, promptly, and elaborates with relevant	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately
-----------------------	----	---	---	------------------------------------	---------------------------------

CO5 – AT3 Mock Interview

Answered Questions

1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Query processing is the process of converting an SQL query into a form that the DBMS can execute efficiently.

The main steps are:

- 1. Parsing and translation** – The DBMS checks the SQL syntax and converts the query into an internal representation such as a query tree.
- 2. Query optimization** – The optimizer considers different ways of executing the query, such as different join orders, indexes, and access methods.
- 3. Execution plan selection** - It estimates the cost of each possible plan using factors such as disk I/O, CPU time, number of rows, and available indexes.
- 4. Query execution** – The selected plan is given to the execution engine, which accesses the required data and produces the result.

The optimizer normally chooses the plan with the lowest estimated cost. For example, if an index can retrieve a small number of matching rows instead of scanning the whole table, the optimizer may choose the index scan.

2. What is the difference between heuristic optimization and cost-based optimization? When would you use each?

Heuristic optimization uses predefined rules to improve a query without calculating the exact cost of every possible plan. Common rules include performing selections early, reducing unnecessary columns, and simplifying operations.

Cost-based optimization estimates the cost of different execution plans using statistics such as table size, number of rows, selectivity, and available indexes. It then chooses the plan with the lowest estimated cost.

I would use heuristic optimization when a quick and simple optimization is needed. Cost-based optimization is more useful for complex queries because it can compare several possible plans and select the one expected to perform best.

In short, heuristic optimization mainly follows rules, while cost-based optimization makes a decision using estimated execution cost.

3. How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

In a real project, I would first create the database and user credentials, then configure the application's database connection settings. I would use a connection library or driver, establish a connection, execute SQL using prepared statements, process the results, and finally close the resources. Connection pooling can be used in larger applications to improve performance.

JDBC stands for Java Database Connectivity. It is a Java API used by Java applications to communicate with databases through JDBC drivers.

ODBC stands for Open Database Connectivity. It is a standard interface that allows applications to access databases using ODBC drivers and is language-independent.

The main difference is that JDBC is designed specifically for Java applications, while ODBC is a general database connectivity standard that can be used by applications written in different programming languages.

4. What are ACID properties? Give a real-world example where each property is critical.

ACID properties ensure that database transactions are reliable and maintain consistent data.

- 1. Atomicity** – A transaction is completed fully or not performed at all. Example: In a bank transfer, money should be deducted from one account and added to another; if one step fails, the whole transaction should be rolled back.
- 2. Consistency** – A transaction must take the database from one valid state to another valid state. Example: A bank account should not violate rules such as an invalid balance caused by a failed transaction.

3. **Isolation** – Concurrent transactions should not incorrectly affect one another. Example: Two customers trying to book the same seat should not both be allowed to reserve it.
4. **Durability** – Once a transaction is committed, its changes should remain even after a system failure. Example: After an online payment is confirmed, the payment record should not disappear after a power failure.

5. Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Concurrency problems can occur when multiple transactions access the same data at the same time.

- **Dirty read** – A transaction reads data written by another transaction before that transaction commits. If the first transaction rolls back, the value that was read was never actually valid.
- **Lost update** – Two transactions read the same value and both update it. One update can overwrite the other, so one change is lost.
- **Unrepeatable read** – A transaction reads the same row twice and gets different values because another transaction changed and committed the row between the two reads.

Two-Phase Locking (2PL) controls access using locks. In the growing phase, a transaction can acquire locks but cannot release them. In the shrinking phase, it can release locks but cannot acquire new ones. By controlling when transactions can read or write shared data, 2PL produces conflict-serializable schedules and helps prevent these concurrency anomalies.

6. What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

A deadlock occurs when two or more transactions wait for each other indefinitely. For example, transaction T1 holds a lock needed by T2, while T2 holds a lock needed by T1.

Deadlock detection can use a wait-for graph:

1. Create a node for every active transaction.
2. Add an edge $T_i \rightarrow T_j$ when T_i is waiting for a resource locked by T_j .
3. Check the graph for a cycle.
4. If a cycle exists, a deadlock is present.
5. The DBMS selects a transaction as a victim, rolls it back, and releases its locks so the other transaction can continue.

Two prevention strategies are:

- **Wait-Die** – An older transaction may wait for a younger transaction, while a younger transaction requesting a lock held by an older one is rolled back.
- **Wound-Wait** – An older transaction can force a younger transaction to roll back; a younger transaction may wait for an older one.

7. Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Immediate Update allows database pages to be written to disk before the transaction commits. Therefore, recovery must be able to undo changes made by transactions that later fail. Log records are maintained so the DBMS can undo uncommitted changes and redo committed changes when required.

Deferred Update does not write a transaction's database changes to the database until the transaction commits. If the transaction fails before commit, its changes do not need to be undone in the database. Recovery mainly needs to redo the committed transactions whose changes were not yet written.

Immediate Update is useful when the system needs flexibility to write updated pages before commit and can support both undo and redo recovery. Deferred Update is simpler conceptually and is useful when changes can be held until commit, although it may require sufficient space or buffering for pending updates.

8. Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Shadow Paging is a recovery technique that keeps two page tables: a shadow page table representing the stable database state and a current page table containing changes made by the transaction.

When a page is modified, the original page is not overwritten. Instead, a new page is written and the current page table is changed to point to it. If the transaction commits, the system updates the pointer to the new page table. If it fails, the shadow page table can be used to return to the previous consistent state.

Advantages:

- It can provide fast recovery because there is no need to scan a long log.
- Rollback can be simple because the shadow page table still represents the old state.
- It avoids overwriting the original pages during the transaction.

Limitations:

- Maintaining page tables can create overhead.
- Copy-on-write can cause fragmentation and additional disk activity.
- It can be less suitable for large, highly concurrent systems.
- Log-based recovery is generally more flexible and widely used in practical DBMSs, especially when supporting complex transactions and high concurrency.

9. What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

A schedule is serializable if its effect is equivalent to executing the transactions serially, one after another, even though their operations may be interleaved.

To test conflict serializability using a precedence graph:

1. Create one node for each transaction.
2. Look for conflicting operations from different transactions. Conflicts occur when the operations access the same data item and at least one operation is a write.
3. If T_i 's conflicting operation occurs before T_j 's operation, draw an edge $T_i \rightarrow T_j$.
4. Check the graph for a cycle.

If the precedence graph has no cycle, the schedule is conflict-serializable. If the graph contains a cycle, the schedule is not conflict-serializable.

10. Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

The main variations of Two-Phase Locking are:

- **Basic 2PL** – A transaction has a growing phase in which it acquires locks and a shrinking phase in which it releases locks. It guarantees conflict serializability but can still allow cascading rollbacks.
- **Strict 2PL** – All exclusive (write) locks are held until the transaction commits or aborts. This prevents other transactions from reading or overwriting uncommitted data and avoids cascading rollback.
- **Rigorous 2PL** – Both shared (read) and exclusive (write) locks are held until commit or abort. It provides an even stronger locking discipline.

- Strict 2PL is commonly used in practical database systems because it provides conflict serializability and makes recovery simpler by preventing other transactions from accessing uncommitted writes. The exact protocol used can vary by DBMS and workload.